Hanin Dhiya Ahmad - 24/534288/TK/59218

PROBLEM 1 - SNIPPSET A - NESTED LOOP

```
int sum = 0
For (int i = 0 ; i < n; i++) {
    for ( int j = 0 ; j < i ; j++) {
        sum += arr [i] + arr [j] ;
    }
}
```

Analisis

Operasi	Cost	Jumlah Execute	Total Execute
int sum = 0	1	1	1
i = 0	1	1	1
i < n	1	$n+1$	$n+1$
i++	1	n	n
j = 0	1	n	n
j < i	1	$\frac{n(n-1)}{2} + n$	$\frac{n(n-1)}{2} + n$
j++	1	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
sum += arr[i] + arr[j]	5	$\frac{n(n-1)}{2}$	$\frac{5n(n-1)}{2}$

(i) Exact Operation Count T(n)

$$T(n) = 1 + 1 + (n+1) + n + n + \left(\frac{n^2-n}{2} + n\right) + \frac{n^2-n}{2} + \frac{5n^2-5n}{2}$$

$$T(n) = \frac{7n^2 - 7n + 8n + 6}{2} \Rightarrow T(n) = 3.5n^2 + 0.5n + 3 \quad //$$

(ii) Big-O Class

$O(n^2)$ Karena T(n) didominasi suku kuadrat, kemudian koef. dihilangkan //

(iii) Best Worst Case

Sama, batas loop (n dan i) sudah ditentukan secara tetap dan tidak bergantung pada nilai data di dalam array. Algoritma ini tidak memiliki cara untuk menghentikan loop. Sehingga apapun isi Array, loop akan tetap berjalan sebanyak n^2.

SNIPSET B

```
int lo = 0, hi = n-1, target = 42 ;
bool found = false ;
while ( lo <= hi ) {
    int mid = lo + (hi - lo) / 2 ;
    if (arr [mid] == target )   { found = true; break; }
    else if ( arr [mid] < target ) { lo = mid + 1 ; }
```

PROBLEM 3B

Operasi	Best Case	Worst Case	Average Case
Linear Search (Unsorted)	O(1) - Target at First Index	O(n) - Target at end or missing	O(n) - Target in middle
Linear Search (Sorted, Early exit)	O(1) - Target at first Index	O(n) - Target > all element	O(n)
Binary Search (Sorted)	O(1) - Target at mid	O(log n) - Pencarian hingga tersisa 1	O(log n)

At what minimum array size n does binary search become more efficient than linear search in the worst case?

↳ Binary search menjadi jauh efisien ketika n > 10. Contoh pada n = 1000, Linear search butuh 1000 langkah untuk kasus terburukanya, sedangkan Binary search hanya butuh ≈ 10 langkah untuk kasus terburuknya. Intinya lebih efisien ketika n yang besar.

PROBLEM 4

PART A

Call #	Value Pushed	Size After	Capacity After	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	1+2 (resize) = 3
4	40	4	4	1
5	50	5	8	1+4 = 5
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	1+8 (resize) = 9
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

PART B - Total Cost

1. Total Cost $(T_{12}) = 1 + 1 + 3 + 1 + 3 + 1 + 1 + 9 + 1 + 1 + 1 = 26$ //
2. Avg Cost $= 26/12 \approx 2.17$ /operasi //
3. Total copy work occurs at sizes. $2, 4, 8, \ldots, 2^k$

   Total Cost $T(n) = n + \sum_{i=0}^{\lfloor \log_2 n \rfloor} 2_i$

   Geometri series sum $\sum_{i=0}^{k} 2^i = 2^{k+1} - 1 \approx 2n$

   Total $T(n) \approx n + 2n = 3n$

   Amortized Cost : $T(n)/n = 3$, its $O(1)$ //

Operasi	Cost	Execute
Inisialisasi (lo, hi, target, found)	4	1
~~Perb~~ while (lo <= hi)	1	$\lceil \log_2 n \rceil + 1$
mid = lo + (hi - lo)/2	4	$\lceil \log_2 n \rceil + 1$
if (arr [mid] = target)	2	$\lceil \log_2 n \rceil + 1$
update lo atau hi	2	$\lceil \log_2 n \rceil + 1$

(i) Operation T(n)

$T(n) \approx 4 + (\log_2 n + 1)(1 + 4 + 2 + 2) + 1$

$T(n) \approx 9 \log_2 n + 14$ //

(ii) Big - O Class

$O(\log n)$ //

(iii) Best Worst Case

Berbeda,

- Best ~~Class~~ Case : $O(1)$ Jika ditemukan pd. percobaan pertama //
- Worst ~~Class~~ Case : $O(\log n)$ jika ditemukan pada akhir proses pembagian //

SNIPPET C : Triple loops

Komponen	Execute
Outer Loop (i)	n
Middle Loop (j)	$\frac{n(n-1)}{2}$
Inner loop (k)	$\lceil n/2 \rceil$

(i) Operation T(n)

$T(n) \approx \frac{n(n-1)}{2} \times \frac{n}{2} \times 7$

$T(n) \approx 1.75 n^3 - 1.75 n^2$ //

(ii) Big - O Class

$O(n^3)$ //

(iii) Best Worst Case

Sama,

kondisi it tidak menghentikan loop. loop tetap melakukan pengecekan sebanyak n^3 //

PART C - Growth Factor Comparison (Fixed Increment $k = 4$)

Start Capacity = 2. New Capacity = old + 4

Resize occur when size hits : 2, 6, 10, 14 ...

1. Trace (fixed Increment)

Call #	Value Pushed	Size After	Capacity After	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	1+2=3
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	7	10	1+6=7
8	80	8	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	1+10=11
12	120	12	14	1

2. Comparison Total Cost

- fixed Increment = 1 + 1 + 3 + 1 + 1 + 1 + 7 + 1 + 1 + 1 + 11 + 1 = 30
- Doubling = 26

Doubling lebih efisien dibanding fixed Increment untuk 12 call ini

3. Derivation T(n)

- Doubling : Seperti Part B, Total time $O(n)$, Amortized $O(1)$
- fixed Increment : Resizes at 2, 6, 10, ..., $4m+2$

  Number of resizes $\approx n/4$

  Cost of resize at step m is proportional to current capacity ($\approx 4m$)

  Total Cost $T(n) = \sum_{m=1}^{n/4} 4m \approx 4 \cdot \frac{(n/4)^2}{2} \approx O(n^2)$
- Amortized Cost : $O(n^2)/n = O(n)$
- Conclusion : Total cost fixed Increment adalah $O(n^2)$ sehingga Amortized Cost adalah $O(n)$. Doubling jauh lebih baik karena menjaga Amortized Cost tetap constan seiring bertambahnya n, sedangkan fixed Increment dengan Amortized Cost Linear.